Zaawansowane techniki sztucznej inteligencji

DOKUMENTACJA PROJEKTU

# Rozpoznawanie kwiatów w czasie rzeczywistym na iPhonie

z wykorzystaniem transfer learningu (MobileNetV3, Core ML)


| | |
|---|---|
| Autor: | **Dawid Stefański** |
| Nr albumu: | **122429** |
| Forma realizacji: | **samodzielna** |
| Data: | **październik 2026** |
| Kod źródłowy: | **Repozytorium** |

# Spis treści


**1. Wstęp . . . 3**

1.1. Cel projektu . . . 3

1.2. Zakres prac . . . 3

**2. Wykorzystane narzędzia . . . 3**

**3. Dane . . . 4**

3.1. Zbiory danych . . . 4

3.2. Podział danych . . . 5

3.3. Przetwarzanie i augmentacja . . . 5

**4. Architektura systemu . . . 5**

4.1. Struktura projektu . . . 6

**5. Modele . . . 6**

5.1. Model bazowy – sieć CNN uczona od zera . . . 6

5.2. MobileNetV3 i transfer learning . . . 7

**6. Przebieg uczenia . . . 7**

6.1. Obserwacje . . . 9

**7. Wyniki . . . 9**

7.1. Porównanie modeli . . . 9

7.2. Wyniki dla poszczególnych klas . . . 10

7.3. Macierz pomyłek . . . 11

7.4. Próg pewności . . . 11

**8. Eksport modelu do Core ML . . . 11**

**9. Aplikacja mobilna . . . 12**

9.1. Jak przetwarzana jest jedna klatka . . . 12

9.2. Co widzi użytkownik . . . 13

9.3. Pliki aplikacji . . . 13

**10. Napotkane problemy i ich rozwiązania . . . 14**

**11. Podsumowanie . . . 14**

11.1. Realizacja kamieni milowych . . . 14

11.2. Wnioski . . . 14

11.3. Ograniczenia i możliwe rozszerzenia . . . 15

**12. Instrukcja uruchomienia . . . 15**

12.1. Część ML (komputer Mac) . . . 15

12.2. Aplikacja (iPhone) . . . 15

# 1. Wstęp

Projekt powstał w ramach przedmiotu *Zaawansowane techniki sztucznej inteligencji*. Punktem wyjścia był temat nr 8 z listy propozycji – **klasyfikator obrazów kwiatów z wykorzystaniem sieci MobileNet**. Temat został rozszerzony o aplikację na iPhone'a, która rozpoznaje kwiaty na żywo z kamery telefonu.

## 1.1. Cel projektu

Celem było zbudowanie modelu sieci neuronowej, który na podstawie zdjęcia rozpoznaje gatunek kwiatu, a następnie uruchomienie go bezpośrednio na telefonie – bez serwera i bez internetu. Główną techniką wykorzystaną w projekcie jest **uczenie transferowe** (ang. *transfer learning*): zamiast uczyć sieć od zera, wykorzystuje się sieć wcześniej wytrenowaną na bardzo dużym zbiorze zdjęć (ImageNet) i dostosowuje ją do własnego, małego zbioru danych.

## 1.2. Zakres prac

- przygotowanie danych – 5 gatunków kwiatów oraz dodatkowa klasa „to nie kwiat”,
- wytrenowanie modelu bazowego (prosta sieć konwolucyjna uczona od zera),
- wytrenowanie modelu MobileNetV3 metodą transfer learningu w dwóch etapach,
- ewaluacja i porównanie modeli na niezależnym zbiorze testowym,
- konwersja najlepszego modelu do formatu Core ML (format modeli Apple),
- aplikacja mobilna rozpoznająca kwiaty w czasie rzeczywistym.

# 2. Wykorzystane narzędzia

| Obszar | Narzędzie | Zastosowanie |
|---|---|---|
| **Język (część ML)** | Python 3.12 | skrypty do danych, treningu i ewaluacji |
| **Uczenie maszynowe** | PyTorch 2.7, torchvision 0.22 | definicja i trening sieci, gotowy model MobileNetV3 |
| **Metryki i wykresy** | scikit-learn, matplotlib | accuracy, F1, macierz pomyłek, krzywe uczenia |
| **Konwersja modelu** | coremltools 9.0 | zamiana modelu PyTorch na Core ML |
| **Aplikacja mobilna** | Swift, SwiftUI, Xcode | interfejs i logika aplikacji |
| **Kamera i obraz** | AVFoundation, Vision | pobieranie klatek z kamery, skalowanie obrazu |
| **Uruchamianie modelu** | Core ML | inferencja na telefonie (Neural Engine) |
| **Sprzęt** | Mac z procesorem Apple M5, iPhone 17 | trening na GPU (MPS), testy aplikacji |

*Tabela 1. Narzędzia i biblioteki użyte w projekcie*

Wersje bibliotek zostały „przypięte” (plik `requirements.txt`). Było to konieczne, ponieważ coremltools 9.0 oficjalnie obsługuje PyTorch tylko do wersji 2.7 – z nowszą wersją konwersja mogła działać niepoprawnie.

# 3. Dane

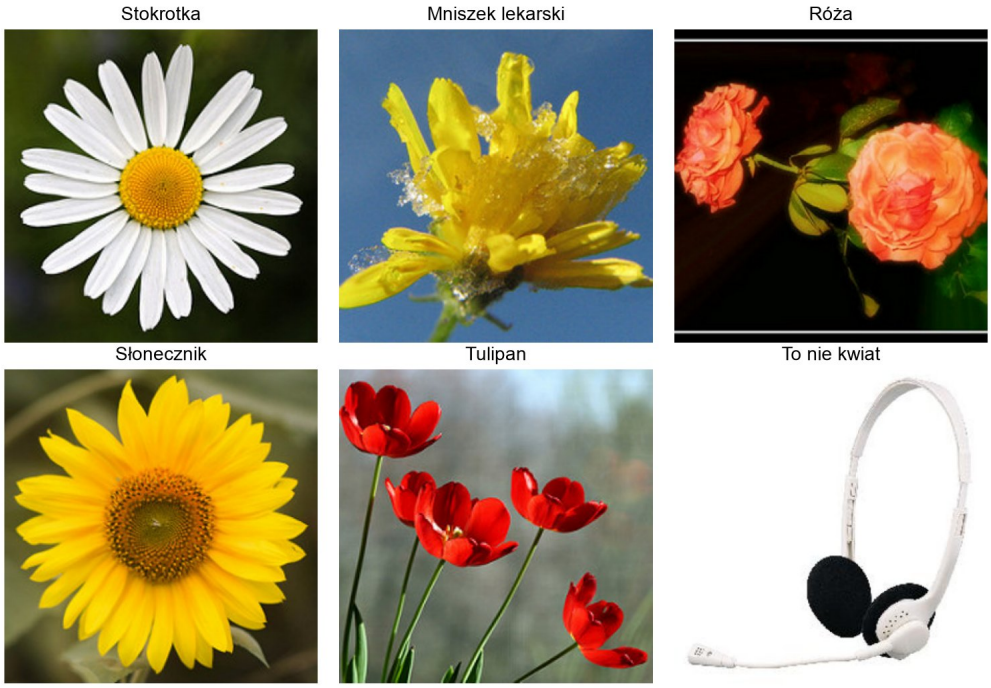
## 3.1. Zbiory danych

Do nauki wykorzystano publiczny zbiór **tf_flowers** (udostępniany przez zespół TensorFlow), który zawiera 3670 zdjęć pięciu gatunków kwiatów. Zdjęcia mają różną rozdzielczość (najczęściej ok. 320×240 – 500×375 pikseli) i pochodzą z serwisu Flickr.

Szybko okazało się, że sam klasyfikator kwiatów nie wystarczy do aplikacji z kamerą: model zawsze wybiera jedną z klas, więc nawet kubek zostałby nazwany np. tulipanem. Dlatego dodano szóstą klasę **„to nie kwiat”**. Zdjęcia do niej pobrano ze zbioru **Caltech-101** – wylosowano po 8 zdjęć z 95 kategorii (przedmioty, zwierzęta, pojazdy). Pominięto kategorie związane z roślinami (słonecznik, lotos, lilia wodna, bonsai, truskawka).

| Klasa | Źródło | Liczba zdjęć |
|---|---|---|
| Stokrotka | tf_flowers | 633 |
| Mniszek lekarski | tf_flowers | 898 |
| Róża | tf_flowers | 641 |
| Słonecznik | tf_flowers | 699 |
| Tulipan | tf_flowers | 799 |
| To nie kwiat | Caltech-101 | 760 |
| **Razem** | | **4430** |

*Tabela 2. Liczba zdjęć w poszczególnych klasach*




*Rysunek 1. Przykładowe zdjęcia z każdej klasy*

Podczas przeglądania danych zauważono, że zbiór tf_flowers nie jest idealny – część zdjęć ma błędne lub wątpliwe etykiety (rys. 2). Nie usuwano ich ręcznie, ale warto o tym pamiętać przy

ocenie wyników: część „błędów” modelu może wynikać z błędów w danych.




*Rysunek 2. Przykłady błędnych lub nietypowych etykiet w zbiorze tf_flowers*

### 3.2. Podział danych

Dane podzielono losowo, ale **stratyfikowanie** (zachowując proporcje klas), na trzy części. Zbiór treningowy służy do nauki, walidacyjny do wyboru najlepszej epoki, a testowy – tylko do końcowej oceny. Podział zapisano do pliku `splits.json` ze stałym ziarnem losowości, więc każde uruchomienie daje te same zbiory.

| Zbiór | Udział | Stokr. | Mniszek | Róża | Słon. | Tulipan | Inne | Razem |
|---|---|---|---|---|---|---|---|---|
| **Treningowy** | 70% | 443 | 629 | 449 | 489 | 559 | 532 | **3101** |
| **Walidacyjny** | 15% | 95 | 134 | 96 | 105 | 120 | 114 | **664** |
| **Testowy** | 15% | 95 | 135 | 96 | 105 | 120 | 114 | **665** |

*Tabela 3. Podział danych na zbiory*

### 3.3. Przetwarzanie i augmentacja

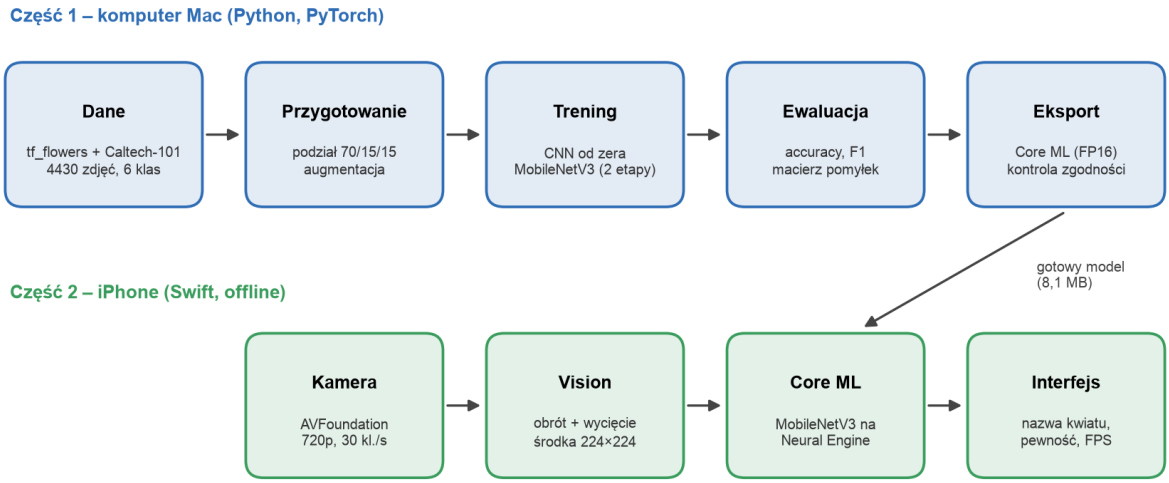
Każde zdjęcie przed podaniem do sieci jest skalowane do rozmiaru **224×224** pikseli i normalizowane (średnia i odchylenie standardowe ze zbioru ImageNet). Podczas treningu stosowano **augmentację**, czyli losowe modyfikacje zdjęć, które sztucznie zwiększają różnorodność danych i naśladują warunki z kamery telefonu:

- losowe kadrowanie (50–100% zdjęcia) i odbicie lustrzane,
- obrót o maksymalnie ±15°,
- zmiana jasności, kontrastu i nasycenia kolorów,
- czasem lekkie rozmycie (symulacja drżenia ręki lub braku ostrości).

Przy ewaluacji augmentacji nie ma – zdjęcie jest tylko skalowane i przycinane do środka, **dokładnie tak samo jak robi to aplikacja na iPhonie**. Dzięki temu wyniki z Pythona dobrze przenoszą się na telefon.

## 4. Architektura systemu

System składa się z dwóch części. Pierwsza działa na komputerze i służy do przygotowania modelu (trening raz, „offline”). Druga to aplikacja na telefonie, która korzysta z gotowego modelu i działa bez internetu.




*Rysunek 3. Schemat działania systemu*

### 4.1. Struktura projektu

| Plik / folder | Opis |
|---|---|
| ml/config.py | wspólne ustawienia: klasy, ścieżki, rozmiar obrazu |
| ml/prepare_data.py | zebranie listy zdjęć i podział na train/val/test |
| ml/data.py | wczytywanie zdjęć, augmentacja, DataLoader |
| ml/models.py | definicje modeli: CNN od zera i MobileNetV3 |
| ml/train.py | pętla treningowa, etapy treningu, zapis najlepszego modelu |
| ml/evaluate.py | metryki, macierz pomyłek, wykresy, raport |
| ml/export_coreml.py | konwersja do Core ML i sprawdzenie zgodności |
| ml/run_all.sh | uruchomienie całego procesu jedną komendą |
| ios/ | aplikacja iOS (projekt Xcode, pliki Swift, model FlowerClassifier.mlpackage) |

*Tabela 4. Najważniejsze pliki projektu*

## 5. Modele

### 5.1. Model bazowy – sieć CNN uczona od zera

Żeby sprawdzić, ile faktycznie daje transfer learning, najpierw wytrenowano prostą sieć konwolucyjną od zera (z losowymi wagami). Sieć ma 5 bloków, a każdy blok to dwie warstwy splotowe 3×3 z normalizacją (BatchNorm) i funkcją ReLU, zakończone warstwą MaxPool. Liczba filtrów rośnie: 32 → 64 → 128 → 256 → 256. Na końcu jest uśrednianie globalne, Dropout (0,3) i warstwa w pełni połączona z 6 wyjściami. Sieć ma ok. **2,36 mln parametrów**.

### 5.2. MobileNetV3 i transfer learning

**MobileNetV3-Large** to sieć zaprojektowana specjalnie z myślą o telefonach – jest mała (4,2 mln parametrów) i szybka, a jednocześnie dokładna. Wykorzystuje m.in. sploty rozdzielne (*depthwise separable convolutions*), które wymagają wielokrotnie mniej obliczeń niż zwykłe sploty. Użyto wersji wytrenowanej wcześniej na zbiorze **ImageNet** (1,28 mln zdjęć, 1000 klas).

Idea transfer learningu jest prosta: wcześniejsze warstwy sieci nauczyły się na ImageNet rozpoznawać ogólne cechy obrazu (krawędzie, tekstury, kształty), które przydają się także do kwiatów. Wystarczy więc wymienić ostatnią warstwę (1000 wyjść → 6 wyjść) i douczyć sieć na własnych danych. Trening przebiegał w dwóch etapach:

- **Etap 1 – zamrożony backbone (8 epok).** Wagi części ekstrahującej cechy są „zamrożone” (nie zmieniają się), uczy się tylko klasyfikator na końcu sieci.
- **Etap 2 – fine-tuning (12 epok).** Odmrażane są górne bloki sieci (od bloku 10) i douczane razem z klasyfikatorem, ale z 10 razy mniejszym współczynnikiem uczenia, żeby nie zepsuć cech wyuczonych na ImageNet.

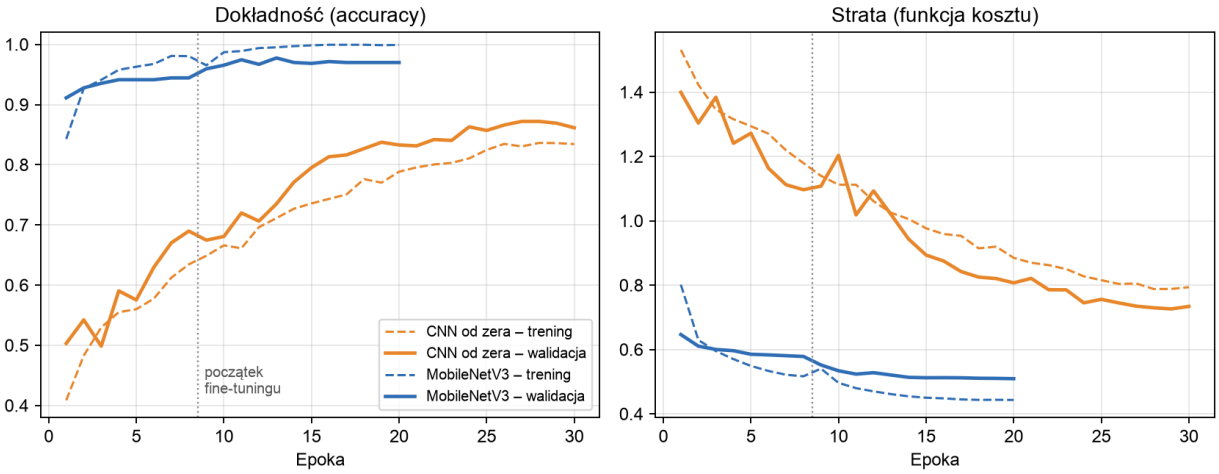
| Parametr | CNN od zera | MobileNetV3 – etap 1 | MobileNetV3 – etap 2 |
|---|---|---|---|
| **Liczba epok** | 30 | 8 | 12 |
| **Współczynnik uczenia (lr)** | 0,001 | 0,001 | 0,0001 |
| **Trenowane parametry** | 2,36 mln (wszystkie) | 1,24 mln z 4,21 mln | 4,05 mln z 4,21 mln |
| **Optymalizator** | AdamW | AdamW | AdamW |
| **Harmonogram lr** | cosinusowy | cosinusowy | cosinusowy |
| **Rozmiar batcha** | 32 | 32 | 32 |
| **Funkcja kosztu** | entropia krzyżowa z label smoothing 0,1 | jak obok | jak obok |

*Tabela 5. Ustawienia treningu (hiperparametry)*

Po każdej epoce model był sprawdzany na zbiorze walidacyjnym i zapisywany był tylko **najlepszy** wariant (o najwyższej dokładności walidacyjnej). Trening odbywał się na GPU komputera Mac (backend MPS w PyTorch).

## 6. Przebieg uczenia

Rysunek 4 pokazuje, jak zmieniała się dokładność i wartość funkcji kosztu w kolejnych epokach. Linie przerywane to zbiór treningowy, linie ciągłe – walidacyjny. Pionowa linia oznacza moment przejścia MobileNetV3 z etapu 1 do etapu 2 (fine-tuning).




*Rysunek 4. Krzywe uczenia obu modeli*

| Epoka | Acc. trening | Acc. walidacja | Strata walid. |
|---|---|---|---|
| 1 | 40,9% | 50,3% | 1,399 |
| 2 | 48,3% | 54,2% | 1,304 |
| 5 | 56,0% | 57,5% | 1,272 |
| 10 | 66,6% | 68,1% | 1,203 |
| 15 | 73,6% | 79,5% | 0,893 |
| 20 | 78,8% | 83,3% | 0,807 |
| 25 | 82,5% | 85,7% | 0,756 |
| 27 (najlepsza) | 83,0% | 87,2% | 0,735 |
| 30 | 83,4% | 86,1% | 0,734 |

*Tabela 6. Postęp uczenia – CNN od zera (wybrane epoki)*

| Epoka | Etap | Acc. trening | Acc. walidacja | Strata walid. |
|---|---|---|---|---|
| 1 | etap 1 | 84,3% | 91,1% | 0,646 |
| 2 | etap 1 | 92,7% | 92,8% | 0,610 |
| 4 | etap 1 | 95,8% | 94,1% | 0,596 |
| 8 | etap 1 | 98,0% | 94,4% | 0,578 |
| 9 | etap 2 | 96,5% | 95,9% | 0,552 |
| 11 | etap 2 | 98,9% | 97,4% | 0,523 |
| 13 (najlepsza) | etap 2 | 99,5% | 97,7% | 0,521 |
| 16 | etap 2 | 99,9% | 97,1% | 0,512 |
| 20 | etap 2 | 99,9% | 97,0% | 0,509 |

*Tabela 7. Postęp uczenia – MobileNetV3 (epoki 1–8: etap 1, epoki 9–20: etap 2)*

### 6.1. Obserwacje

- **CNN od zera uczy się powoli.** Zaczyna od ok. 50% dokładności walidacyjnej i przez 30 epok stopniowo dochodzi do 87,2% (najlepsza epoka 27). Krzywa jeszcze lekko rośnie, więc dłuższy trening dałby trochę więcej, ale kosztem czasu – cały trening trwał ok. 19 minut (ok. 39 s na epokę).
- **Ciekawostka: w CNN od zera dokładność walidacyjna jest wyższa niż treningowa.** To nie błąd – podczas treningu działają augmentacja i Dropout, które utrudniają zadanie, a przy walidacji ich nie ma.
- **MobileNetV3 już po pierwszej epoce** (ok. 20 s) osiąga 91,1% – więcej niż CNN od zera po 30 epokach. To najlepiej pokazuje siłę transfer learningu: sieć „zna" już ogólne cechy obrazów.
- **Etap 1 szybko się wypłaszcza** na poziomie ok. 94,4%. Zamrożone cechy z ImageNet mają swoje ograniczenia – nie są dopasowane do kwiatów.
- **Fine-tuning daje wyraźny skok:** już w pierwszej epoce etapu 2 dokładność rośnie do 95,9%, a najlepszy wynik to 97,7% (epoka 13).
- **Pod koniec widać początek przeuczenia (overfittingu):** dokładność treningowa dochodzi do 99,9%, a walidacyjna stoi w miejscu ok. 97%. Model zaczyna „zapamiętywać" zbiór treningowy. Przed pogorszeniem wyniku chroni zapisywanie najlepszej epoki.
- **Strata nie spada do zera** (minimum ok. 0,44), bo zastosowano *label smoothing* – model nie jest „nagradzany" za stuprocentową pewność. Dzięki temu jest mniej „przemądrzały" i jego pewność lepiej oddaje rzeczywistość.
- **Czas:** cały trening MobileNetV3 (oba etapy) trwał ok. 6 minut, czyli ponad 3 razy krócej niż CNN od zera, mimo że daje dużo lepszy wynik.

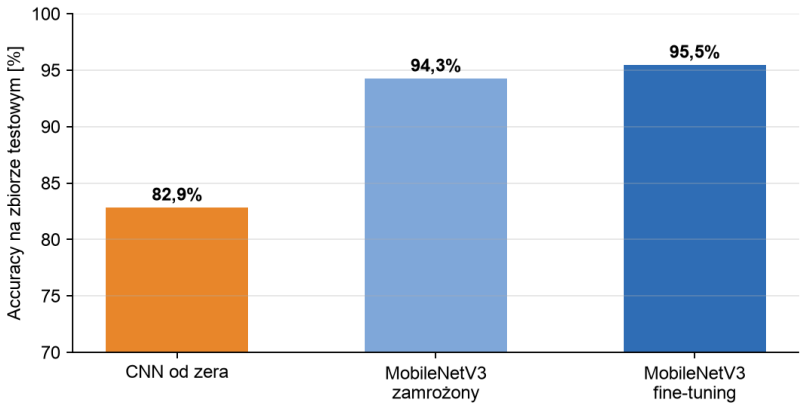
## 7. Wyniki

Wszystkie wyniki poniżej policzono na **zbiorze testowym** (665 zdjęć), którego model nie widział podczas treningu ani przy wyborze najlepszej epoki.

### 7.1. Porównanie modeli

| Model | Accuracy | Macro F1 | Parametry | Czas treningu |
|---|---|---|---|---|
| CNN od zera | 82,9% | 0,823 | 2,36 mln | ok. 19 min |
| MobileNetV3 – etap 1 (zamrożony) | 94,3% | 0,942 | 4,21 mln | ok. 2 min |
| **MobileNetV3 – etap 2 (fine-tuning)** | **95,5%** | **0,953** | 4,21 mln | ok. 6 min (łącznie) |

*Tabela 8. Wyniki modeli na zbiorze testowym*




*Rysunek 5. Dokładność modeli na zbiorze testowym*

**Accuracy** to odsetek poprawnie rozpoznanych zdjęć, **Macro F1** to średnia miary F1 ze wszystkich klas – uwzględnia zarówno precyzję, jak i czułość i nie faworyzuje liczniejszych klas, Transfer learning poprawił wynik o **11,4 punktu procentowego**, a fine-tuning dołożył kolejne **1,2 p.p.**

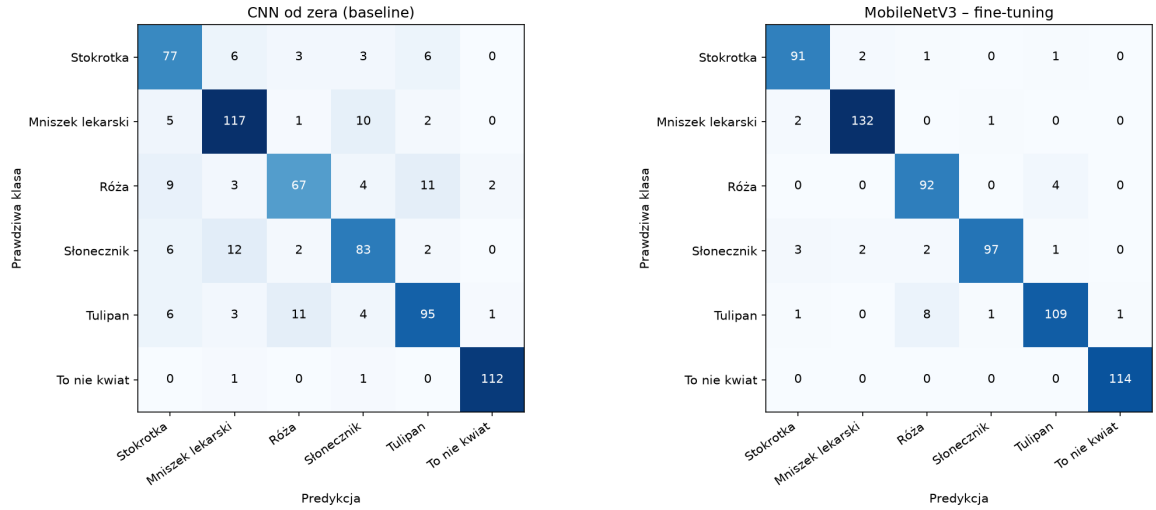
## 7.2. Wyniki dla poszczególnych klas

| Klasa | Precyzja | Czułość | F1 | F1 (CNN od zera) | Liczba zdjęć |
|---|---|---|---|---|---|
| **Stokrotka** | 0,938 | 0,958 | 0,948 | 0,778 | 95 |
| **Mniszek lekarski** | 0,971 | 0,978 | 0,974 | 0,845 | 135 |
| **Róża** | 0,893 | 0,958 | 0,925 | 0,744 | 96 |
| **Słonecznik** | 0,980 | 0,924 | 0,951 | 0,790 | 105 |
| **Tulipan** | 0,948 | 0,908 | 0,928 | 0,805 | 120 |
| **To nie kwiat** | 0,991 | 1,000 | 0,996 | 0,978 | 114 |

*Tabela 9. Precyzja, czułość i F1 dla każdej klasy (MobileNetV3 po fine-tuningu)*

**Precyzja** mówi, jaka część zdjęć oznaczonych przez model jako dana klasa naprawdę do niej należy. **Czułość** (recall) – jaką część zdjęć danej klasy model znalazł. Najlepiej rozpoznawany jest mniszek i klasa „to nie kwiat”, najsłabiej róża i tulipan.

## 7.3. Macierz pomyłek




*Rysunek 6. Macierze pomyłek: CNN od zera (po lewej) i MobileNetV3 po fine-tuningu (po prawej). Wiersze – prawdziwa klasa, kolumny – odpowiedź modelu.*

Na przekątnej macierzy są poprawne odpowiedzi, poza nią – pomyłki. Najczęstszy błąd MobileNetV3 to **mylenie tulipana z różą** (8 tulipanów uznanych za róże i 4 róże za tulipany). Jest to zrozumiałe – oba kwiaty bywają czerwone, a pąki mają podobny kształt. Klasa „to nie kwiat" została rozpoznana bezbłędnie (114/114). W CNN od zera pomyłek jest wyraźnie więcej i są rozłożone między wszystkie gatunki.

## 7.4. Próg pewności

Model zwraca dla każdej klasy prawdopodobieństwo. W aplikacji nazwa kwiatu jest pokazywana tylko wtedy, gdy pewność przekracza ustalony **próg** – w przeciwnym razie pojawia się komunikat „Nie jestem pewien". Tabela 10 pokazuje, jak próg wpływa na działanie:

| Próg | Kwiaty, dla których pokazano wynik | Trafność pokazanych wyników | Nie-kwiaty uznane za kwiat |
|---|---|---|---|
| 50% | 93,5% | 97,5% | 0,0% |
| 60% | 89,1% | 98,6% | 0,0% |
| 70% | 84,2% | 99,4% | 0,0% |
| 80% | 75,7% | 99,8% | 0,0% |
| 90% | 43,9% | 100,0% | 0,0% |

*Tabela 10. Wpływ progu pewności na wyniki (zbiór testowy)*

Wyższy próg oznacza mniej pomyłek, ale częściej odpowiedź „nie wiem". Jako domyślny wybrano **70%**: aplikacja podaje wynik dla 84% zdjęć kwiatów i ma wtedy rację w 99,4% przypadków. W aplikacji próg można zmieniać suwakiem.

# 8. Eksport modelu do Core ML

Model z PyTorcha nie może być bezpośrednio użyty na iPhonie, dlatego został przekonwertowany do formatu **Core ML** za pomocą biblioteki coremltools. Przy konwersji:

- normalizację obrazu i funkcję softmax wbudowano w model – aplikacja podaje surową klatkę z kamery i od razu dostaje prawdopodobieństwa klas,
- wagi zapisano w precyzji **FP16** (16-bitowej) – model jest 2 razy mniejszy, a układ Neural Engine w iPhonie działa natywnie w tej precyzji,
- w model wpisano nazwy klas, dzięki czemu Core ML zwraca gotowe etykiety.

Ponieważ konwersja i zmiana precyzji mogą zmienić wyniki, przeprowadzono **test zgodności**: każde zdjęcie testowe sklasyfikowano oryginalnym modelem i modelem Core ML, a następnie porównano odpowiedzi.

| Sprawdzana rzecz | Wynik |
|---|---|
| Ta sama odpowiedź (klasa) w PyTorch i Core ML | 665 z 665 zdjęć (100,0%) |
| Accuracy modelu Core ML na zbiorze testowym | 95,49% |
| Największa różnica prawdopodobieństwa | 0,017 |
| Rozmiar modelu (FlowerClassifier.mlpackage) | 8,1 MB |
| Czas jednej predykcji na Macu – tylko CPU | 1,93 ms |
| Czas jednej predykcji na Macu – z Neural Engine | 0,42 ms |

*Tabela 11. Wyniki testu zgodności modelu Core ML*

Model Core ML daje dokładnie te same odpowiedzi co oryginał, więc wyniki z rozdziału 7 obowiązują także dla aplikacji.

# 9. Aplikacja mobilna

Aplikacja została napisana w języku Swift z użyciem SwiftUI. Po uruchomieniu pokazuje obraz z tylnej kamery, a na dole ekranu kartę z wynikiem rozpoznawania. Wszystko dzieje się na telefonie – obraz nigdzie nie jest wysyłany.

## 9.1. Jak przetwarzana jest jedna klatka

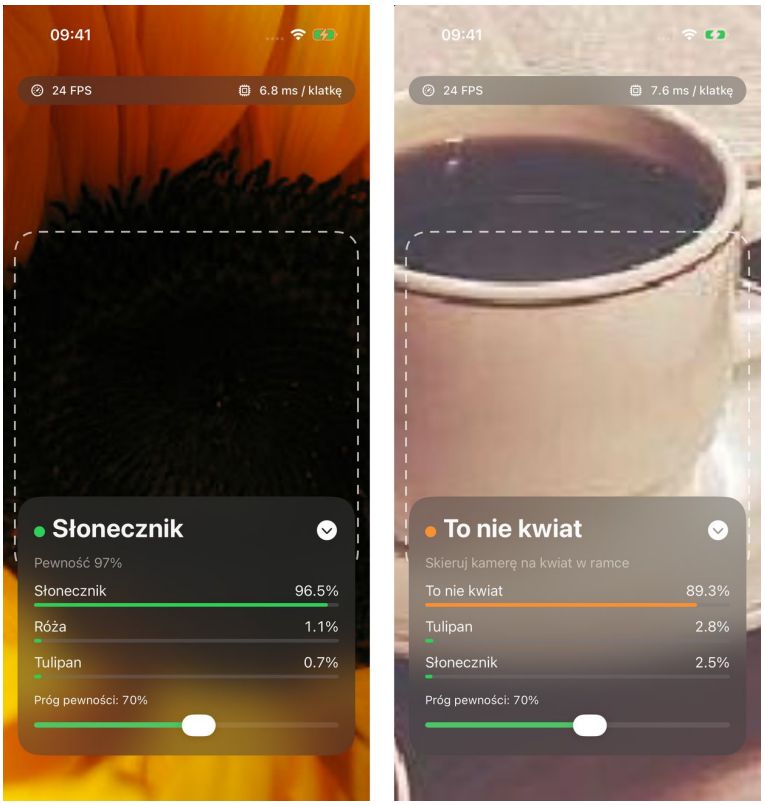
1. Kamera (AVFoundation) dostarcza klatkę obrazu 1280×720 ok. 30 razy na sekundę.
2. Biblioteka Vision obraca klatkę do pionu i wycina z niej środkowy kwadrat, skalując go do 224×224 pikseli.
3. Core ML uruchamia model MobileNetV3 – zwykle na układzie Neural Engine – i zwraca prawdopodobieństwa 6 klas.
4. Wyniki są **wygładzane w czasie** (średnia z poprzednich klatek), żeby odpowiedź nie „skakała” przy każdym drgnięciu ręki.
5. Interfejs pokazuje wynik, trzy najbardziej prawdopodobne klasy oraz liczbę przetwarzanych klatek na sekundę (FPS) i czas jednej predykcji.

Jeśli model nie nadąża z przetwarzaniem, starsze klatki są pomijane – dzięki temu obraz i wynik nie mają narastającego opóźnienia. Na iPhonie 17 licznik w aplikacji pokazuje **ok. 30 klatek na sekundę**, czyli tyle, ile dostarcza kamera – model przetwarza każdą klatkę na bieżąco.

## 9.2. Co widzi użytkownik

| Sytuacja | Komunikat | Kolor |
|---|---|---|
| Model rozpoznał kwiat z pewnością ≥ progu | nazwa kwiatu i pewność, np. „Słonecznik 97%” | zielony |
| Najbardziej prawdopodobna jest klasa „to nie kwiat” | „To nie kwiat” | pomarańczowy |
| Pewność poniżej progu | „Nie jestem pewien” | żółty |

*Tabela 12. Możliwe odpowiedzi aplikacji*




*Rysunek 7. Ekran aplikacji: rozpoznany słonecznik (po lewej) i kubek rozpoznany jako „to nie kwiat” (po prawej). Zrzuty wykonano w symulatorze iPhone'a, gdzie zamiast obrazu z kamery podano zdjęcia ze zbioru testowego; widoczne wartości FPS i ms pochodzą z symulatora, a nie z telefonu.*

## 9.3. Pliki aplikacji

| Plik | Odpowiedzialność |
|---|---|
| CameraManager.swift | obsługa kamery: uprawnienia, konfiguracja, pobieranie klatek |
| CameraPreview.swift | podgląd obrazu z kamery na ekranie |
| FlowerRecognizer.swift | uruchamianie modelu (Vision + Core ML), wygładzanie wyników, pomiar FPS |
| ContentView.swift | interfejs: ramka celownika, karta z wynikiem, suwak progu |
| FlowerClassifier.mlpackage | wytrenowany model (Xcode sam generuje do niego klasę Swift) |

*Tabela 13. Pliki aplikacji iOS*

# 10. Napotkane problemy i ich rozwiązania

| Problem | Rozwiązanie |
|---|---|
| Najnowszy PyTorch (2.14) nie był przetestowany z coremltools – biblioteka ostrzegała o możliwych błędach konwersji. | Zainstalowano PyTorch 2.7.0 (ostatnia wersja wspierana przez coremltools 9.0) i przypięto wersje w requirements.txt. |
| Klasyfikator z 5 klasami zawsze wybierał jakiś kwiat – nawet dla zdjęcia kubka czy ściany. | Dodano szóstą klasę „to nie kwiat” (zdjęcia z Caltech-101) oraz próg pewności w aplikacji. |
| Skrypt treningowy na macOS kończył pracę, ale proces się nie zamykał (zawieszał się przy wyjściu). | Przyczyną były procesy pomocnicze PyTorcha (torch_shm_manager) tworzone przez równoległe wczytywanie danych. Ustawiono wczytywanie w jednym procesie – zdjęcia są małe, więc trening zwolnił tylko nieznacznie. |
| Pomiar czasu w PyTorch na CPU pokazywał, że MobileNetV3 jest wolniejszy niż CNN od zera, mimo ok. 13 razy mniejszej liczby obliczeń. | PyTorch na CPU słabo optymalizuje sploty rozdzielne. Ten pomiar usunięto z raportu – miarodajny jest czas w Core ML, który na telefonie i tak jest jedynym istotnym. |
| Zbiór tf_flowers zawiera błędne etykiety (rys. 2). | Nie poprawiano ich ręcznie; uwzględniono to przy interpretacji wyników. |

*Tabela 14. Problemy napotkane podczas realizacji*

# 11. Podsumowanie

## 11.1. Realizacja kamieni milowych

| Kamień milowy | Kryterium | Wynik | Status |
|---|---|---|---|
| **KM1. Dane i model bazowy** | podział 70/15/15, wytrenowany model bazowy | 82,9% | zrealizowany |
| **KM2. Model z transfer learningiem** | accuracy ≥ 90% i macro F1 ≥ 0,90 | 95,5% / 0,953 | zrealizowany |
| **KM3. Eksport do Core ML** | zgodność ≥ 99%, rozmiar ≤ 20 MB | 100%, 8,1 MB | zrealizowany |
| **KM4. Aplikacja w czasie rzeczywistym** | ≥ 15 FPS, < 30 ms na klatkę, działanie offline | ok. 30 FPS na iPhonie 17 | zrealizowany |

*Tabela 15. Stan realizacji kamieni milowych*

## 11.2. Wnioski

- Transfer learning bardzo dobrze sprawdza się przy małym zbiorze danych – MobileNetV3 osiągnął 95,5% dokładności, podczas gdy sieć uczona od zera 82,9%, a trening trwał przy tym ponad 3 razy krócej.
- Fine-tuning (odmrożenie części warstw) daje dodatkowy zysk w porównaniu z samym douczeniem klasyfikatora, ale trzeba uważać na przeuczenie.
- Do aplikacji działającej „na żywo” sama dokładność nie wystarczy – potrzebna była klasa „to nie kwiat”, próg pewności i wygładzanie wyników w czasie.
- Model po konwersji do Core ML i zmianie precyzji na FP16 zachował dokładnie te same odpowiedzi, a zajmuje tylko 8,1 MB.

### 11.3. Ograniczenia i możliwe rozszerzenia

- Model zna tylko 5 gatunków – każdy inny kwiat zostanie przypisany do najbardziej podobnego albo oznaczony jako niepewny. Rozszerzeniem byłby większy zbiór, np. Oxford 102 Flowers.
- Zdjęcia z Caltech-101 są dość „czyste” (pojedynczy obiekt), więc wynik 100% dla klasy „to nie kwiat” jest optymistyczny – prawdziwe klatki z kamery bywają trudniejsze.
- Zdjęcia z internetu różnią się od obrazu z kamery telefonu (światło, rozmycie, kadr). Augmentacja częściowo to kompensuje, ale warto byłoby douczyć model na własnych zdjęciach.
- Możliwe dodatki: rozpoznawanie zdjęć z galerii, wizualizacja Grad-CAM (na co „patrzy” sieć), kwantyzacja wag do 8 bitów.

## 12. Instrukcja uruchomienia

### 12.1. Część ML (komputer Mac)

```
cd ml
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -r requirements.txt
# dane: data/flower_photos (tf_flowers) i data/101_ObjectCategories (Caltech-101)
./run_all.sh      # podzial danych, trening, ewaluacja, eksport (ok. 25 min)
```

Wyniki (raport, wykresy, model Core ML) pojawiają się w folderze `ml/outputs/`, a model jest automatycznie kopiowany do projektu aplikacji.

### 12.2. Aplikacja (iPhone)

1. Otworzyć w Xcode projekt aplikacji z folderu `ios/`.
2. Podłączyć iPhone'a, wybrać go jako urządzenie docelowe i nacisnąć **Run**.
3. Przy pierwszym uruchomieniu zaufać certyfikatowi dewelopera w ustawieniach telefonu (Ustawienia → Ogólne → VPN i urządzenia) i zezwolić na dostęp do kamery.